Learning Mím and MímIR

A syntax guide and study workbook for compiler engineers

Prepared from the MímIR repository

September 2026

Contents

1 How to use this guide **3**

2 The core mental model **3**

3 Lexical syntax **4**
3.1 Files, tokens, and comments . . . 4
3.2 Unicode and ASCII spellings . . . 4
3.3 Literals . . . 4

4 Modules, imports, and plugins **5**
4.1 Imports . . . 5
4.2 Plugins . . . 5
4.3 Modules and `use` . . . 5
4.4 Visibility . . . 6

5 Names, bindings, and declaration expressions **6**
5.1 `let` . . . 6
5.2 Declaration expressions . . . 6
5.3 Rebinding . . . 6
5.4 `where` . . . 7

6 Types and universes **7**
6.1 Built-in type-level forms . . . 7
6.2 Function types . . . 7
6.3 Implicit parameters . . . 7

7 Patterns and telescopes **8**
7.1 Patterns use parentheses . . . 8
7.2 Telescopes use brackets . . . 8
7.3 Grouped names . . . 8

8 Functions and continuations **8**
8.1 Direct-style functions: `lam` and `λ` . . . 8
8.2 Continuations: `con`, `cn`, and `Cn` . . . 9
8.3 Returning CPS functions: `fun`, `fn`, and `Fn` . . . 9
8.4 Application . . . 9
8.5 `ret` . . . 9
8.6 Partial-evaluation filters . . . 9

9 Products, arrays, and updates **10**
9.1 Tuple and Sigma syntax . . . 10
9.2 Extraction . . . 10
9.3 Functional insertion . . . 10
9.4 Arrays and packs . . . 10

10 Unions and pattern matching **11**

11 Records and recursion **11**
11.1 Recursive type declarations . . . 11
11.2 Recursive functions . . . 11

12 Externals, annexes, and axioms **11**
12.1 extern . . . 11
12.2 Annexes . . . 12
12.3 Axioms . . . 12

13 Rewrite rules **12**

14 Operator precedence **12**

15 Worked example: SSA expressed as CPS **13**

16 Worked example: polymorphism and partial evaluation **14**

17 Worked example: values inside types **15**

18 From source syntax to graph nodes **15**

19 Compilation and inspection **15**
19.1 Typical build . . . 15
19.2 Running tests . . . 16
19.3 Inspecting a Mím file . . . 16

20 Common mistakes **16**
20.1 Confusing Mím with MímIR . . . 16
20.2 Using brackets where a body needs bindings . . . 16
20.3 Treating . as field extraction . . . 16
20.4 Forgetting that operators are ordinary names . . . 16
20.5 Expecting every declaration to survive . . . 17
20.6 Treating insertion as mutation . . . 17
20.7 Shadowing the mem module . . . 17
20.8 Assuming lexical nesting controls placement . . . 17
20.9 Confusing Nat, Idx n, and I32 . . . 17

21 Four-week study plan **17**
21.1 Week 1: read and write basic Mím . . . 17
21.2 Week 2: graph semantics and evaluation . . . 17
21.3 Week 3: CPS and phases . . . 18
21.4 Week 4: plugins and lowering . . . 18

22 Exercises **19**
22.1 Syntax exercises . . . 19
22.2 Graph exercises . . . 19
22.3 CPS exercise . . . 19
22.4 Dependent-type exercise . . . 20
22.5 Plugin exercise . . . 20

23 Quick-reference sheet **20**

24 Glossary **21**

25 Where to continue **21**

1 How to use this guide

This guide teaches the textual **Mím** language and connects each construct to the graph-based **MímIR** it creates. It assumes familiarity with MLIR, SSA, compiler passes, and ordinary typed IRs.

Terminology matters:

- **Mím** is the textual language stored in .mim files.
- **MímIR** is the graph IR and its C++/Python API.
- **mim** is the command-line compiler.
- **Plugin annexes** extend the deliberately small built-in language.

The best way to study is to type and run the examples rather than merely read them. After each chapter, complete the short exercises before continuing.

2 The core mental model

Mím source resembles a small functional language, but its let bindings and lexical layout are mostly presentation. Compilation constructs a graph of Def nodes in a World.

Every relevant entity is a graph node:

- terms and values;
- types and type-level computations;
- functions and applications;
- variables and binders;
- tuples, arrays, and literals;
- plugin primitives;
- compiler phases.

Most nodes are immutable and hash-consed. Structurally identical immutable nodes are represented by the same pointer. Binders are mutable when variables, recursion, or cyclic structure require the compiler to create the binder before its body.

The fundamental built-in families are:

Type former	Introduced by	Used by
Pi (function type)	Lam	App
Sigma (product type)	Tuple	Extract / Insert
Arr (array type)	Pack	Extract / Insert

Types and terms inhabit the same graph. A value can influence a type, a function can return a type, and type-level applications are normalized by the same machinery as ordinary applications.

3 Lexical syntax

3.1 Files, tokens, and comments

Mím files are UTF-8 and contain a sequence of declarations. Statements are normally terminated with semicolons.

```
// A normal line comment.
/* A non-nesting block comment. */
/// Documentation forwarded to generated plugin documentation.
```

Identifiers start with a letter or underscore. Backticks turn an operator into an ordinary identifier:

```
let `+ = core.nat.add;
let five = 2 + 3;
```

The operator expression a + b is syntax sugar for:

```
`+ (a, b)
```

Consequently, operators have no intrinsic arithmetic meaning. Their meaning is determined by whichever declaration is bound to `+, `-, and so on.

3.2 Unicode and ASCII spellings

Mím uses Unicode as its primary notation, with ASCII alternatives for common tokens:

Primary	ASCII alternative	Meaning
→	->	function arrow
←	<-	functional aggregate insertion
λ	lm	anonymous direct-style lambda
⊥	bot	bottom/non-returning type or literal
⊤	top	top literal

This guide uses ASCII arrows in larger examples so they are easy to type, but repository code normally prefers the primary UTF-8 syntax.

3.3 Literals

Unannotated integer literals default to Nat:

```
let n = 42;          // Nat
```

Fixed-width integer literals may use an explicit type:

```
let x = 42:I32;
let y = 42I32;       // preferred shorthand
let z = 0xffI16;
```

Mím's fixed-width integer types are finite index types:

```
I32 == Idx i32
Bool == Idx i1
tt  == 1_2
ff  == 0_2
```

Idx n has n inhabitants. Thus Idx 3 contains the indices 0, 1, and 2. i32 denotes the cardinality (2^{32}), while I32 denotes the corresponding index type.

Characters and strings use familiar syntax:

```
let c = 'x';
let s = "MimIR";
```

+ and - belong to expressions rather than numeric tokens. Parenthesize a negative argument:

```
f (-23)
```

Without the parentheses, f -23 parses as subtraction.

4 Modules, imports, and plugins

4.1 Imports

Every file is a module.

```
import foo;
import "relative/path/bar.mim";
import foo as renamed;
import foo as *;
```

import foo searches for foo.mim and binds the module as foo. as * imports public members into the current scope without binding the module name itself.

An imported file is parsed, bound, and emitted only once.

4.2 Plugins

```
plugin core;
plugin core as *;
```

plugin core does two things:

1. loads the core shared library;
2. imports its matching core.mim interface.

Ordinary import performs only the second kind of operation and never loads a shared library.

A plugin consists of:

- a .mim interface declaring annexes;
- a native library registering normalizers and compiler phases.

4.3 Modules and use

```
mod arithmetic {
    pub let answer = 42;
}

use arithmetic as a;
let x = a.answer;
```

use path; is shorthand for use path as *;.

Paths use dots:

```
core.nat.add
mem.load
compile.beta_red
```

A dot selects a module member. It does **not** extract a field from a runtime value; field extraction uses #.

4.4 Visibility

Declarations are priv by default. pub makes a declaration accessible through a module path.

```
priv let implementation_detail = 1;
pub  let interface_value       = 2;
```

extern and anx normally imply public visibility unless explicitly combined with priv.

5 Names, bindings, and declaration expressions

5.1 let

let names an expression:

```
let x = 21;
let y = core.nat.add (x, x);
```

At the IR level, the name usually disappears. Both uses refer directly to the same graph node.

Patterns can destructure values:

```
let (x, y) = pair;
let (head, tail) as whole = value;
```

The alias pattern binds both the components and the complete value.

5.2 Declaration expressions

Declarations may appear before the final result of an expression:

```
let x = 20;
let y = 22;
core.nat.add (x, y)
```

The value of the complete expression is its final expression.

5.3 Rebinding

let and ret may shadow an earlier name. This is convenient for state threading:

```
let (m, ptr) = mem.alloc (I32, 0) m;
let m        = mem.store (m, ptr, 23I32);
let (m, val) = mem.load (m, ptr);
```

Do not name the state value mem, because that would hide the mem module.

5.4 where

where attaches declarations after the expression that uses them:

```
f 42 where
    lam f (x: Nat): Nat = x;
end
```

This is surface-level lexical organization. A function written in a where block is not physically contained by another node in the resulting MímIR graph. Its dependencies determine logical nesting.

6 Types and universes

6.1 Built-in type-level forms

```
Univ
Type level
*
□
Nat
Idx
Rule T
```

- Univ is the type of universe levels.
- Type e is the universe at level e.
- * abbreviates Type (0:Univ).
- □ abbreviates Type (1:Univ).
- Nat is the unbounded natural-number type.
- Idx has type Nat -> *.
- Rule T is the type of rewrite rules over meta-type T.

For ordinary programming, read * as "the universe of ordinary types."

6.2 Function types

An ordinary function type is:

```
Nat -> Nat
```

A dependent function type names an argument that may appear in the result:

```
[n: Nat] -> «n; I32»
```

This is a Pi type. The result array length is the input value n.

6.3 Implicit parameters

Curly braces mark implicit parameters:

```
lam id {T: *} (x: T): T = x;
let n = id 42;
```

The compiler infers T = Nat. Use @ to provide an implicit argument explicitly:

```
let n = id @Nat 42;
```

7 Patterns and telescopes

Patterns and telescopes look similar but serve different purposes.

7.1 Patterns use parentheses

A pattern receives or destructures an actual value for a body:

```
lam swap {A B: *} (a: A, b: B): [B, A] = (b, a);
```

(a: A, b: B) binds a and b in the function body.

7.2 Telescopes use brackets

A telescope describes a type, with names visible to components on its right:

```
[n: Nat, values: «n; I32»]
```

This is a dependent Sigma type.

An unnamed pattern component must still use _:

```
lam second (_: Nat, y: Nat): Nat = y;
```

An unnamed telescope component is simply a type:

```
[Nat, Nat]
```

7.3 Grouped names

Several names may share one annotation:

```
lam add (x y: Nat): Nat = core.nat.add (x, y);
```

This abbreviates:

```
lam add (x: Nat, y: Nat): Nat = core.nat.add (x, y);
```

8 Functions and continuations

Mím exposes direct style, raw CPS, and return-continuation sugar.

8.1 Direct-style functions: lam and λ

A named direct-style function:

```
lam id {T: *} (x: T): T = x;
```

An anonymous direct-style function:

```
let id = λ {T: *} (x: T): T = x;
```

lm is the ASCII spelling of λ.

Named declarations are recursive because the function name is visible in its own body. Anonymous lambdas are not automatically recursive.

8.2 Continuations: con, cn, and Cn

A continuation does not return to its caller:

```
con done (x: I32) = another_continuation x;
```

Its type can be written:

```
Cn I32
```

which is shorthand for:

```
I32 -> bot
```

con declares a named continuation, while cn constructs an anonymous one.

8.3 Returning CPS functions: fun, fn, and Fn

fun gives CPS code a direct-looking return syntax:

```
fun choose (condition: Bool, x y: I32): I32 =
    core.select (condition, x, y);
```

Conceptually, the compiler adds a return continuation:

```
choose(condition, x, y, return)
```

The type-level form is:

```
Fn [Bool, I32, I32] -> I32
```

8.4 Application

Application is juxtaposition:

```
f x
f x y
f (x, y)
```

MímIR functions internally have one domain. Apparent multiple arguments are represented through currying or a Sigma/tuple domain.

8.5 ret

ret binds the result of a continuation-style call:

```
ret result = computation $ argument;
use result
```

The $ separates the continuation-style callee from its argument.

8.6 Partial-evaluation filters

A domain may carry a filter after @:

```
lam specialize (n: Nat)@(core.pe.is_closed n): Nat = body;
```

The filter controls whether applying the corresponding lambda may beta-reduce during graph construction.

The usual direct-style default is tt, allowing eager reduction. The final domain of con, cn, fun, and fn defaults to ff, preserving control-flow continuations rather than blindly inlining them.

9 Products, arrays, and updates

9.1 Tuple and Sigma syntax

Tuple values use parentheses:

```
let point = (10I32, 20I32);
```

Tuple/Sigma types use brackets:

```
let Point = [I32, I32];
```

A dependent Sigma type may name fields:

```
rec Vector = [n: Nat, data: «n; I32»];
```

9.2 Extraction

Extract by numeric index:

```
let x = point#0;
```

Extract by a named field:

```
let n = vector#n;
```

Field names from mutable Sigma types take precedence after #. Parenthesize an ordinary index variable when necessary:

```
value#(i)
```

9.3 Functional insertion

Insertion creates a new aggregate; it does not mutate the original:

```
let moved = point#0 <- 30I32;
```

Nested insertion updates the full path:

```
let updated = matrix#i#j <- value;
```

9.4 Arrays and packs

Array types:

```
«4; I32»
«i: n; ElementType i»
«i: rows, j: cols; I32»
```

Corresponding pack values:

```
‹4; 0I32›
‹i: n; make i›
```

A homogeneous literal tuple and equivalent pack may canonicalize to the same IR representation.

10 Unions and pattern matching

A union type combines alternatives:

```
A ∪ B
```

Inject a value into a union:

```
value inj A ∪ B
```

Eliminate the union using match:

```
match value with
| left  => handle_left left
| right => handle_right right
```

The exact patterns must agree with the union representation and expected type.

11 Records and recursion

11.1 Recursive type declarations

rec creates a mutable recursive Sigma or function type:

```
rec Node = [value: Nat, next: Option Node];
```

The declaration name is available while constructing its body, allowing the graph to contain a cycle.

Mutual recursive declarations use and:

```
rec A = ...
and B = ...;
```

11.2 Recursive functions

Named lam, con, and fun declarations are recursive:

```
lam factorial (n: Nat): Nat =
   // recursive use of factorial is legal here
   ...;
```

Mutually recursive functions also use and.

12 Externals, annexes, and axioms

12.1 extern

An external declaration is a program root:

```
extern fun main(argc: I32): I32 = return argc;
```

Roots survive cleanup and are visible to backends.

A bodyless external declaration describes a symbol implemented elsewhere:

```
extern lam native_function [I32]: I32;
```

12.2 Annexes

anx registers a declaration in the global annex table:

```
anx lam plugin.operation (x: Nat): Nat = x;
```

Tools such as compile.named can find it through its plugin-qualified name.

12.3 Axioms

axm declares a typed primitive without a Mím body:

```
axm demo.const_idx: [n: Nat] -> Idx n, normalize_const;
```

The optional tail names a C++ normalizer and may include curry and trip counters. Plugin C++ code supplies the behavior.

An axiom family can share one type:

```
axm nat.(add, sub, mul): «2; Nat» -> Nat, normalize_nat;
```

Normalizers execute eagerly when sufficiently applied axiom nodes are constructed. They implement constant folding, canonicalization, and local semantic simplification.

13 Rewrite rules

Mím supports rule and normalizing norm declarations:

```
rule example pattern: MetaType
when condition
=> replacement;
```

norm declares a rule intended to participate in normalization.

When designing rules, consider:

- whether they terminate;
- whether competing rules are confluent;
- whether matching preserves types;
- whether a rewrite duplicates expensive subgraphs.

14 Operator precedence

From strongest to weakest, the practically important levels are:

1. typed literals;
2. extraction with #;
3. application and explicit implicit application;
4. shifts;
5. multiplication, division, and remainder;
6. addition and subtraction;

7. relational comparisons;
8. equality comparisons;
9. function arrows;
10. unions;
11. injection;
12. insertion;
13. where.

Application binds very tightly:

```
f x + g y
```

means:

```
(f x) + (g y)
```

Function arrows associate to the right:

```
A -> B -> C
```

means:

```
A -> (B -> C)
```

There are no special grouping expressions. A one-element tuple degrades to its element, which provides the expected parenthesized behavior.

15 Worked example: SSA expressed as CPS

The following is adapted from lit/docs/count.mim:

```
plugin core;

use core.ops.u.w;

extern fun count(n: I32): I32 =
    loop (0I32, 0I32)
    where
        con loop (i acc: I32) =
            core.select ((i < n), body, exit) ()
            where
                con body() =
                    loop (i + 1I32, acc + i);
                con exit() =
                    return acc;
            end;
    end;
```

Read it as follows:

1. core provides machine arithmetic, comparisons, and select.
2. use core.ops.u.w binds unsigned wrapping operators such as + and <.
3. count is a returning CPS function.
4. loop is a continuation corresponding to an SSA loop header.
5. i and acc are block arguments/phi results.
6. body and exit are continuations corresponding to successor blocks.
7. Selecting a continuation and applying it performs a conditional branch.
8. Calling loop recursively is the back edge.

9. return acc invokes the hidden return continuation of count.

The SSA correspondence is:

Conventional SSA	Mím/CPS
basic block	con continuation
block argument / phi result	continuation parameter
branch	continuation call
conditional branch	select a continuation, then call it
back edge	recursive continuation call
return	call the return continuation

Unlike an MLIR region, the lexical where structure does not own the continuation nodes. Free-variable dependencies determine where the backend may schedule them.

16 Worked example: polymorphism and partial evaluation

The following is adapted from lit/docs/iter.mim:

```
plugin core;
plugin refly;

use core.ops.n;

extern lam iter {T: *} (f: T -> T)
            (n: Nat, x: T)@(core.pe.is_closed n): T =
   core.select ((n <= 0), cons, alt) ()
   where
      lam cons(): T = x;

      lam alt(): T =
         let m = n - 1;
         let y = f x;
         iter @T f (m, y);
   end;

lam succ (x: Nat): Nat = x + 1;
lam add  (x: Nat) (y: Nat): Nat = iter succ (x, y);
lam mul  (x: Nat) (y: Nat): Nat = iter (add x) (y, 0);
lam pow  (x: Nat) (y: Nat): Nat = iter (mul x) (y, 1);

let _ = refly.equiv.struc_eq (pow 3 5, 243);
```

Important observations:

- T is an implicit type argument.
- f is a first-class function.
- iter is higher-order and parametrically polymorphic.
- iter @T explicitly supplies the inferred type argument in the recursive call.
- the filter permits unrolling when n is closed;
- partial application produces functions such as add x;
- pow 3 5 normalizes to 243 during graph construction;
- refly.equiv.struc_eq checks structural equality at compile time.

17 Worked example: values inside types

```
lam Vec (n: Nat): * = «n; Nat»;

lam zeros (n: Nat): Vec n =
    ‹n; 0›;

let three_zeroes = zeros 3;
```

Vec is an ordinary function returning a type. Applying Vec n beta-reduces to «n; Nat».

The same n variable influences:

- the result type of zeros;
- the length of the returned pack.

This is the central dependent-type property of MímIR: a term-level graph node can be an operand of a type-level graph node.

18 From source syntax to graph nodes

Use the following map while reading compiler dumps:

Mím construct	Typical MímIR representation
lam / λ	Lam with a Pi type
con / cn	Lam whose Pi codomain is bottom
fun / fn	returning continuation-form Lam
application	App
[A, B]	Sigma
(a, b)	Tuple
«n; T»	Arr
‹n; v›	Pack
x#i	Extract
x#i <- v	Insert
numeric value	Lit
bound identifier	Var pointing to a binder
axm	Axm plus applications
unresolved inference value	mutable Hole
recursive declaration	mutable binder
let name	usually no node of its own

19 Compilation and inspection

19.1 Typical build

```
cmake -S . -B build \
    -DCMAKE_BUILD_TYPE=Debug \
    -DBUILD_TESTING=ON \
    -DMIM_BUILD_EXAMPLES=ON

cmake --build build
```

19.2 Running tests

```
cmake --build build --target test-all
cmake --build build --target lit
ctest --test-dir build --output-on-failure
```

The lit suite uses a staged copy. Rebuild its staging target after editing a test, or use the repository's single-test probing workflow.

19.3 Inspecting a Mím file

Useful output modes include textual dumps, S-expressions, and Graphviz DOT. Consult the current command-line help because exact combinations evolve:

```
build/bin/mim --help
```

When reading a graph:

- boxes are Def nodes;
- edges point from a node to its operands;
- immutable nodes are structurally shared;
- mutable binders may participate in cycles;
- type edges may be hidden in simplified diagrams;
- externals and annexes form roots;
- cleanup discards unreachable subgraphs.

20 Common mistakes

20.1 Confusing Mím with MímIR

let, module paths, and where blocks belong to the textual frontend. Do not assume they correspond to owned regions or explicit nodes in MímIR.

20.2 Using brackets where a body needs bindings

Incorrect:

```
lam f [I32, I32]: I32 = ...
```

Use a pattern:

```
lam f (_: I32, x: I32): I32 = x;
```

Brackets describe a type; parentheses destructure a value for a body.

20.3 Treating . as field extraction

```
module.member   // module path
value#field     // value extraction
```

20.4 Forgetting that operators are ordinary names

Import or bind the intended operator family before writing x + y.

20.5 Expecting every declaration to survive

Only definitions reachable from externals or annexes survive cleanup. Mark an entry point extern when it must be emitted.

20.6 Treating insertion as mutation

x#i <- value creates a new aggregate. It does not modify x.

20.7 Shadowing the mem module

Use m for the memory-state value so mem.load and mem.store remain visible.

20.8 Assuming lexical nesting controls placement

where improves readability but does not create MLIR-like region ownership. Free variables and binder nesting determine placement.

20.9 Confusing Nat, Idx n, and I32

- Nat is an unbounded natural number.
- Idx n is a finite index in [0,n).
- I32 is Idx 2^32, used as a 32-bit machine integer.

21 Four-week study plan

21.1 Week 1: read and write basic Mím

Study:

- literals and types;
- modules and plugins;
- let, patterns, and telescopes;
- direct-style functions;
- tuples, arrays, extraction, and insertion.

Practice:

1. Write polymorphic id.
2. Write swap for a heterogeneous pair.
3. Write a function returning ‹n; 0›.
4. Use a named Sigma field and extract it.
5. Dump each program and identify Pi, Lam, App, Sigma, and Tuple.

Repository reading:

- docs/langref.md;
- lit/sigma.mim;
- lit/pack.mim;
- examples/poly.cpp.

21.2 Week 2: graph semantics and evaluation

Study:

- immutable versus mutable definitions;
- hash-consing;
- beta reduction;
- free variables;
- roots and cleanup;
- axioms and normalizers.

Practice:

1. Predict which lambdas disappear after beta reduction.
2. Write repeated expressions and inspect whether they share nodes.
3. Create an unused non-external function and observe cleanup.
4. Examine src/mim/plug/demo/.
5. Trace demo.const_idx from .mim declaration to C++ normalizer.

Repository reading:

- docs/mimir.md;
- docs/dev.md;
- include/mim/def.h;
- include/mim/world.h;
- src/mim/plug/demo/.

21.3 Week 3: CPS and phases

Study:

- con, Cn, fun, and Fn;
- SSA/CPS correspondence;
- continuation parameters as block arguments;
- Phase, Analysis, RWPhase, and fixed points.

Practice:

1. Rewrite an if/else as two continuations.
2. Rewrite a loop as a recursive continuation.
3. Explain each edge in the count graph.
4. Build and run the SCCP examples.
5. Trace how optimize(World&) locates _compile or _default_compile.

Repository reading:

- lit/docs/count.mim;
- docs/phases.md;
- examples/sccp_analysis.cpp;
- examples/sccp_transform.cpp;
- src/mim/phase/optimize.cpp.

21.4 Week 4: plugins and lowering

Study:

- annex generation;
- plugin loading;
- phase registration;
- memory-state threading;
- closure conversion;
- default lowering pipeline;
- LLVM emission.

Practice:

1. Add a second constant-like operation to the demo plugin.
2. Give it a constant-folding normalizer.
3. Write a lit test for the operation.
4. Trace a small program through opt.mim.
5. Identify the point where high-level functions become backend-compatible code.

Repository reading:

- docs/plugins.md;
- src/mim/plug/compile/compile.mim;
- src/mim/plug/opt/opt.mim;
- src/mim/plug/mem/mem.mim;
- src/mim/plug/clos/;
- src/mim/plug/ll/ll.cpp.

22 Exercises

22.1 Syntax exercises

1. Define const with type {A B: *} -> A -> B -> A.
2. Define a dependent pair containing n: Nat and «n; I32».
3. Construct a value of that type for $n = 4$.
4. Replace the second element of $(1, 2, 3)$ without mutation.
5. Write a function with an explicit implicit type application.

22.2 Graph exercises

For each expression, draw the expected shared graph:

```
let x = f a;
g (x, x)
```

```
lam id {T: *} (x: T): T = x;
id 42
```

Identify:

- shared nodes;
- binder-variable edges;
- type edges;
- nodes that normalize away.

22.3 CPS exercise

Translate this pseudocode into fun and con declarations:

```
sum(n):
  i = 0
  acc = 0
  while i < n:
    acc = acc + i
    i = i + 1
  return acc
```

The loop continuation should take i and acc as parameters.

22.4 Dependent-type exercise

Write:

```
replicate : [T: Type, n: Nat] -> T -> Array n T
```

Then construct four booleans and verify the result's type.

22.5 Plugin exercise

Starting from src/mim/plug/demo/:

1. declare a typed axiom;
2. generate its C++ annex identifier;
3. register a normalizer;
4. return a canonical literal when all required inputs are known;
5. add a lit test proving the simplification occurred.

23 Quick-reference sheet

```
// Plugins and modules
plugin core;
import foo;
use core.ops.n;

// Values and products
let x = 42;
let pair = (x, x);
let first = pair#0;
let changed = pair#1 <- 23;

// Types
let Pair = [Nat, Nat];
let Vec  = «4; Nat»;
let Bool = Idx i1;

// Direct style
lam id {T: *} (x: T): T = x;
let answer = id @Nat 42;

// Continuations
con next (x: I32) = done x;
let Next = Cn I32;

// Returning CPS
fun f (x: I32): I32 = return x;
let F = Fn I32 -> I32;

// Arrays and packs
let zero_type = «n; I32»;
let zero_pack = ‹n; 0I32›;

// External root
extern fun main(argc: I32): I32 = return argc;
```

```
// Local declarations
f x where
    lam f (y: Nat): Nat = y;
end
```

24 Glossary

Annex A globally registered entity exported by a plugin.
Axiom A typed primitive with no ordinary Mím definition.
Beta reduction Applying a lambda by substituting its argument into the body.
Binder A node that introduces a variable, such as a lambda or dependent type.
Bottom ⊥/bot; the result type of a computation that does not return normally.
CPS Continuation-passing style, where the rest of a computation is represented by a function.
Def The base class of every MímIR graph node.
Dependent type A type containing or computed from a value.
External A rooted definition visible to backends and preserved by cleanup.
Free variable A variable used by an expression but not bound by that expression.
Hash-consing Reusing one immutable node for structurally identical constructions.
Normalizer Plugin code that simplifies an axiom application during graph construction.
Pack The value-level counterpart of an array type.
Phase A whole-program compiler transformation or analysis.
Pi type A possibly dependent function type.
Root A graph entry from which reachability and liveness are determined.
Sigma type A possibly dependent product or record type.
Telescope An ordered sequence of binders where later types may depend on earlier values.
World The owner, factory, and canonicalization context for a complete MímIR graph.

25 Where to continue

Read the repository documentation in this order:

1. docs/README.md
2. docs/mimir.md
3. docs/langref.md
4. docs/dev.md
5. docs/rewriting.md
6. docs/phases.md
7. docs/plugins.md
8. docs/python.md

Then study:

1. examples/poly.cpp
2. examples/hello.cpp
3. src/mim/plug/demo/
4. examples/sccp_analysis.cpp
5. src/mim/plug/opt/opt.mim

The language reference remains the source of truth for exact syntax. This guide is organized for learning and deliberately introduces constructs in conceptual rather than grammar-production order.